

Workgroup: fapi
Published: 22 February 2025
Status: Final
Author: D. Fett
Authlete

FAPI 2.0 Attacker Model

Abstract

OIDF FAPI 2.0 is an API security profile suitable for high-security applications based on the OAuth 2.0 Authorization Framework [RFC6749]. This document describes that attacker model that informs the decisions on security mechanisms employed by the FAPI security profiles.

Foreword

The OpenID Foundation (OIDF) promotes, protects and nurtures the OpenID community and technologies. As a non-profit international standardizing body, it is comprised by over 160 participating entities (workgroup participant). The work of preparing implementer drafts and final international standards is carried out through OIDF workgroups in accordance with the OpenID Process. Participants interested in a subject for which a workgroup has been established have the right to be represented in that workgroup. International organizations, governmental and non-governmental, in liaison with OIDF, also take part in the work. OIDF collaborates closely with other standardizing bodies in the related fields.

Final drafts adopted by the Workgroup through consensus are circulated publicly for the public review for 60 days and for the OIDF members for voting. Publication as an OIDF Standard requires approval by at least 50% of the members casting a vote. There is a possibility that some of the elements of this document may be subject to patent rights. OIDF shall not be held responsible for identifying any or all such patent rights.

Introduction

Since OIDF FAPI 2.0 aims at providing an API protection profile for high-risk scenarios, clearly defined security requirements are indispensable. In this document, the security requirements are expressed through security goals and attacker models. From these requirements, the security mechanisms utilized in the Security Profile are derived.

Implementers and users of the Security Profile can derive from this document which threats have been taken into consideration by the Security Profile and which fall outside of what the Security Profile provides.

A systematic definition of security requirements and an attacker model enable proofs of the security of the FAPI 2.0 Security Profile, similar to the proofs in [arXiv.1901.11520] for FAPI 1.0, which this work draws from. Formal proofs can rule out large classes of attacks rooted in the logic of security protocols.

The formal analysis performed on this attacker model and the FAPI 2.0 Security Profile, described in [Section 9](#), has helped to refine and improve this document and the FAPI 2.0 Security Profile.

Notational conventions

The keywords "shall", "shall not", "should", "should not", "may", and "can" in this document are to be interpreted as described in ISO Directive Part 2 [ISODIR2]. These keywords are not used as dictionary terms such that any occurrence of them shall be interpreted as keywords and are not to be interpreted with their natural language meanings.

Table of Contents

1. [Scope](#)
2. [Normative references](#)
3. [Terms and definitions](#)
4. [Symbols and Abbreviated terms](#)
5. [Security goals](#)
 - 5.1. [General](#)
 - 5.2. [Authorization](#)
 - 5.3. [Authentication](#)
 - 5.4. [Session integrity](#)
6. [Attacker model](#)
7. [Attackers](#)
 - 7.1. [General](#)
 - 7.2. [A1 - Web attacker](#)
 - 7.3. [A1a - Web Attacker \(participating as authorization server\)](#)
 - 7.4. [A2 - Network attacker](#)
 - 7.5. [Attacker at the authorization endpoint: A3a - read authorization request](#)
 - 7.6. [Attacker at the token endpoint: A4 - read and tamper with token requests and responses](#)
 - 7.7. [Attacker at the resource server: A5 - read resource requests](#)
8. [Limitations](#)
 - 8.1. [General](#)
 - 8.2. [Protocol layers](#)
 - 8.3. [Secrets](#)
 - 8.4. [System boundaries](#)
 - 8.5. [Implementation errors](#)
 - 8.6. [Changes over time](#)

[9. Formal Analysis](#)[10. Security Considerations](#)[11. Normative References](#)[12. Informative References](#)[Appendix A. Acknowledgements](#)[Appendix B. Notices](#)[Author's Address](#)

1. Scope

This document describes the FAPI 2.0 profiles security goals, attacker model, attacker roles and capabilities, and limitations.

2. Normative references

The following documents are referred to in the text in such a way that some or all of their content constitutes requirements of this document. For dated references, only the edition cited applies. For undated references, the latest edition of the referenced document (including any amendments) applies.

See Section 11 for normative references.

3. Terms and definitions

For the purpose of this document, the terms defined in [\[RFC6749\]](#), and [\[OIDC\]](#) apply.

4. Symbols and Abbreviated terms

API – Application Programming Interface

CSRF – Cross-Site Request Forgery

DNS – Domain Name System

JWKS – JSON Web Key Sets

OIDF – OpenID Foundation

TLS – Transport Layer Security

URL – Uniform Resource Locator

5. Security goals

5.1. General

In the following, the security goals for the FAPI 2.0 Security Profile with regards to authorization and, when OpenID Connect is used, authentication, are defined.

5.2. Authorization

The FAPI 2.0 Security Profile aims to ensure that **no attacker can access protected resources** other than their own.

The access token is the ultimate credential for access to resources in OAuth. Therefore, this security goal is fulfilled if no attacker can successfully obtain and use an access token for access to protected resources other than their own.

5.3. Authentication

The FAPI 2.0 Security Profile aims to ensure that **no attacker is able to log in at a client under the identity of another user**.

The ID token is the credential for authentication in OpenID Connect. This security goal therefore is fulfilled if no attacker can obtain and use an ID token identifying another user for login.

5.4. Session integrity

Session integrity is concerned with attacks where a user is tricked into logging in under the attacker's identity or inadvertently using the resources of the attacker instead of the user's own resources. Attacks in this field include CSRF attacks (traditionally defended against by using "state" in OAuth) and session swapping attacks.

In detail:

- For authentication: The FAPI 2.0 Security Profile aims to ensure that **no attacker is able to force a user to be logged in under the identity of the attacker**.
- For authorization: The FAPI 2.0 Security Profile aims to ensure that **no attacker is able to force a user to use resources of the attacker**.

6. Attacker model

This attacker model defines very broad capabilities for attackers. It is assumed that attackers will exploit these capabilities to come up with attacks on the security goals defined above. To provide a very high level of security, attackers are assumed very powerful, including having access to otherwise encrypted communication.

This model does intentionally not define concrete threats. For example, an attacker that has the ability to eavesdrop on an authorization request might be able to use this capability for various types of attacks posing different threats, e.g., injecting a modified authorization request. In a complex protocol like OAuth or OpenID Connect, however, yet unknown types of threats and variants of existing threats can emerge, as has been

shown in the past. In order to not overlook any potential attacks, FAPI 2.0 therefore aims not to address concrete, narrow threats, but to exclude any attacks conceivable for the attacker types listed here. This is supported by a formal security analysis, see [Section 9](#).

This attacker model assumes that certain parts of the infrastructure and protocols are working correctly. Failures in these parts likely lead to attacks that are out of the scope of this attacker model. These areas need to be analyzed separately within the scope of an application of the FAPI 2.0 security profiles using threat modelling or other techniques.

For example, if a major flaw in TLS was found that undermines data integrity in TLS connections, a network attacker (A2, below) would be able to compromise practically all OAuth and OpenID Connect sessions in various ways. This would be fatal, as even application-level signing and encryption is based on key distribution via TLS connections. As another example, if a human error leads to the disclosure of secret keys for authentication and an attacker would be able to misuse these credentials, this attack would not be covered by this attacker model.

The following parts of the infrastructure are out of the scope of this attacker model:

- **TLS:** It is assumed that TLS connections are not broken, i.e., data integrity and confidentiality are ensured. The correct public keys are used to establish connections and private keys are not known to attackers (except for explicitly compromised parties).
- **JWKS:** Where applicable, key distribution mechanisms work as intended, i.e., encryption and signature verification keys of uncompromised parties are retrieved from the correct endpoints.
- **Browsers and endpoints:** Devices and browsers used by resource owners are considered not compromised. Other endpoints not controlled by an attacker behave according to the protocol.
- **Identity and session management:** End user's identity proofing, authentication, identity and access management on a client or authorization server are out of scope for this specification. It is assumed that clients ensure that sessions of different users are properly protected from each other and from attackers. Clients retrieving identity attributes using OpenID Connect are required to check whether the identity attributes returned fulfills their requirements.

7. Attackers

7.1. General

FAPI 2.0 profiles aim to ensure the security goals listed above for arbitrary combinations of the following attackers, potentially collaborating to reach a common goal:

7.2. A1 - Web attacker

This is the standard web attacker model. The attacker:

- can send and receive messages just like any other party controlling one or more endpoints on the internet,
- can participate in protocols flows as a normal user,
- can use arbitrary tools (e.g., browser developer tools, custom software, local interception proxies) on their own endpoints to tamper with messages and assemble new messages,
- can send links to honest users that are then visited by these users.

This means that the web attacker has the ability to cause arbitrary requests from users' browsers, as long as the contents are known to the attacker.

The attacker cannot intercept or block messages sent between other parties, and cannot break cryptography unless the attacker has learned the respective decryption keys. Deviating from the common web attacker model, A1 cannot play the role of a legitimate authorization server in the ecosystem (see A1a).

7.3. A1a - Web Attacker (participating as authorization server)

This is a variant of the web attacker A1, but this attacker can also participate as an authorization server in the ecosystem.

Note that this authorization server can reuse/replay messages it has received from honest authorization servers and can send users to endpoints of honest authorization servers.

7.4. A2 - Network attacker

This attacker controls the whole network (like a rogue WiFi access point or any other compromised network node). This attacker can intercept, block, and tamper with messages intended for other people, but cannot break cryptography unless the attacker has learned the respective decryption keys.

Note: Most attacks that are exclusive to this kind of attacker can be defended against by using transport layer protection like TLS.

7.5. Attacker at the authorization endpoint: A3a - read authorization request

This attacker is assumed to have the capabilities of the web attacker, but it can also read the authorization request sent in the front channel from a user's browser to the authorization server.

This might happen on mobile operating systems (where apps can register for URLs), on all operating systems through the browser history, or due to cross-site scripting on the authorization server. There have been cases where anti-virus software intercepts TLS connections and stores/analyzes URLs.

Note: An attacker that can read the authorization response is not considered here, as, with current browser technology, such an attacker can undermine most security protocols. This is discussed in "Browser Swapping Attacks" in the security considerations in the FAPI 2.0 Security Profile.

Note: The attackers for the authorization request are more fine-grained than those for the token endpoint and resource endpoint, since these messages pass through the complex environment of the user's browser/app/OS with a larger attack surface. This demands for a more fine-grained analysis.

Note: For the authorization and resource endpoints, it is assumed that the attacker can only passively read messages, whereas for the token endpoint, it is assumed that the attacker can also tamper with messages. The underlying assumption is that leakages from the authorization request or response are very common in practice and leakages of the resource request are possible, but a fully compromised connection to either endpoint is very unlikely. In particular for the authorization endpoint, a fully compromised connection would undermine the security of most redirect-based authentication/authorization schemes, including OAuth.

7.6. Attacker at the token endpoint: A4 - read and tamper with token requests and responses

This attacker makes the client use a token endpoint that is not the one of the honest authorization server. This attacker therefore can read and tamper with messages sent to and from this token endpoint that the client thinks as of an honest authorization server.

Important: This attacker is a model for misconfigured token endpoint URLs that were considered in FAPI 1.0. Since the FAPI 2.0 Security Profile mandates that the token endpoint address is obtained from an authoritative source and via a protected channel, i.e., through OAuth metadata obtained from the honest authorization server, this attacker is not relevant in FAPI 2.0. The description here is kept for informative purposes only.

7.7. Attacker at the resource server: A5 - read resource requests

This attacker has the capabilities of the web attacker, but it can also read requests sent to the resource server after they have been processed by the resource server, for example because the attacker can read TLS intercepting proxy logs on the resource server's side.

Note: An attacker that can read the responses from the resource server is not considered here, as such an attacker would directly contradict the authorization goal stated above. If it could tamper with the responses, it could additionally trivially break the session integrity goal.

8. Limitations

8.1. General

Beyond the limitations already described in the introduction to the attacker model above, it is important to note the following limitations:

8.2. Protocol layers

FAPI 2.0 profiles only define the behavior of API authorization and authentication on certain protocol layers. As described above, attacks on lower protocol layers (e.g., TLS) can break the security of FAPI 2.0 compliant systems under certain conditions. The attacker model, however, takes some breaks in the end-to-end security provided by TLS into account by already including the respective attacker models (A3a/A5/A7). Similarly, many other attacks on lower layers are already accounted for, for example:

- DNS spoofing attacks are covered by the network attacker (A2)
- Leakages of authorization request data, e.g., through misconfigured URLs or system/firewall logs, are covered by A3a
- Directing users to malicious websites is within the capabilities of the web attacker (A1)

FAPI 2.0 aims to be secure when attackers exploit these attacks and all attacks feasible to attackers described above, even in combination.

Other attacks are not covered by the attacker model. For example, user credentials being exposed through misconfigured databases or remote code execution attacks on authorization servers are neither prevented by nor accounted for in the attacker model. As another example, when a user is using a compromised browser and operating system, the security of the user is hard to uphold. Phishing-resistant credentials, for example, can help in this case, but are outside of the area defined by FAPI 2.0, as described next.

8.3. Secrets

The security assessment assumes that secrets are created such that attackers cannot guess them - e.g., nonces and secret keys. Weak random number generators, for example, can lead to secrets that are guessable by attackers and therefore to vulnerabilities.

8.4. System boundaries

The FAPI 2.0 profiles focus on core aspects of the API security and do not prescribe, for example, end-user authentication mechanisms, firewall setups, software development practices, or security aspects of internal architectures. Anything outside of boundaries of FAPI 2.0 must be assessed in the context of the ecosystem, deployment, or implementation in which FAPI 2.0 is used.

8.5. Implementation errors

API security profiles define how authentication and authorization are supposed to be implemented and a formal model assesses whether the profiles are secure and consistent with respect to ideal implementations. Real-world implementations, of course, sometimes deviate from the specified and formally analyzed behavior and contain security vulnerabilities on various levels. While the FAPI 2.0 profiles are designed to provide multiple layers of defense where feasible, implementations must use secure software development and deployment best practices to ensure that vulnerabilities can be discovered and fixed.

8.6. Changes over time

New technologies or changed behavior of components (e.g., browsers) can lead to new security vulnerabilities over time that might not have been known during the development of these specifications.

9. Formal Analysis

The FAPI 2.0 Security Profile is accompanied by a formal security analysis [analysis.FAPI2] that provides a formal model of the FAPI 2.0 Security Profile and a proof of the security of the FAPI 2.0 Security Profile within this model. The formal model is based on the attacker model and security goals defined in this document.

Note that the analysis is based on a prior version the attacker model that used a different numbering for the attackers. Some of the attacker models previously considered were in contradiction with the security goals and therefore removed. The mapping between the attacker model in this document and the one used in the analysis is as follows:

Analysis	This document
A1	A1
A1a	A1a
A2	A2
A3a	A3a
A3b	removed - see note in Section 7.5
A5	A4
A7	A5 - with reduced capabilities, see note in Section 7.7
A8	removed - see note in Section 7.7

Table 1

As the updates to the attacker model were made to align with the formal analysis, the analysis results are still valid for the updated attacker model.

10. Security Considerations

This entire document consists of security considerations.

11. Normative References

- [OIDC] Sakimura, N., Bradley, J., Jones, M., de Medeiros, B., and C. Mortimore, "OpenID Connect Core 1.0 incorporating errata set 1", 8 November 2014, <http://openid.net/specs/openid-connect-core-1_0.html>.
- [RFC6749] Hardt, D., Ed., "The OAuth 2.0 Authorization Framework", RFC 6749, DOI 10.17487/RFC6749, October 2012, <<https://www.rfc-editor.org/info/rfc6749>>.

12. Informative References

- [ISODIR2] ISO/IEC, "ISO/IEC Directives, Part 2 - Principles and rules for the structure and drafting of ISO and IEC documents", <<https://www.iso.org/sites/directives/current/part2/index.xhtml>>.
- [analysis.FAPI2] Hosseyni, P., Küsters, R., and T. Würtele, "Formal Security Analysis of the OpenID FAPI 2.0: Accompanying a Standardization Process", 1 October 2022, <https://openid.net/wordpress-content/uploads/2022/12/Formal-Security-Analysis-of-FAPI-2.0_FINAL_2022-10.pdf>.
- [arXiv.1901.11520] Fett, D., Hosseyni, P., and R. Küsters, "An Extensive Formal Security Analysis of the OpenID Financial-grade API", arXiv 1901.11520, 31 January 2019, <<http://arxiv.org/abs/1901.11520/>>.

Appendix A. Acknowledgements

This document was developed by the OpenID FAPI Working Group.

We would like to thank Dave Tonge, Nat Sakimura, Brian Campbell, Torsten Lodderstedt, Joseph Heenan, Pedram Hosseyni, Ralf Küsters and Tim Würtele for their valuable feedback and contributions that helped to evolve this document.

Appendix B. Notices

Copyright (c) 2025 The OpenID Foundation.

The OpenID Foundation (OIDF) grants to any Contributor, developer, implementer, or other interested party a non-exclusive, royalty free, worldwide copyright license to reproduce, prepare derivative works from, distribute, perform and display, this Implementers Draft, Final Specification, or Final Specification Incorporating Errata Corrections solely for the purposes of (i) developing specifications, and (ii) implementing Implementers Drafts, Final Specifications, and Final Specification Incorporating Errata Corrections based on such documents, provided that attribution be made to the OIDF as the source of the material, but that such attribution does not indicate an endorsement by the OIDF.

The technology described in this specification was made available from contributions from various sources, including members of the OpenID Foundation and others. Although the OpenID Foundation has taken steps to help ensure that the technology is available for distribution, it takes no position regarding the validity or scope of any intellectual property or other rights that might be claimed to pertain to the implementation or use of the technology described in this specification or the extent to which any license under such rights might or might not be available; neither does it represent that it has made any independent effort to identify any such rights. The OpenID Foundation and the contributors to this specification make no (and hereby expressly disclaim any) warranties (express, implied, or otherwise), including implied warranties of merchantability, non-infringement, fitness for a particular purpose, or title, related to this specification, and the entire risk as to implementing this specification is assumed by the implementer. The OpenID Intellectual Property Rights policy (found at openid.net) requires contributors to offer a patent promise not to assert certain patent claims against other contributors and against implementers. OpenID invites any interested party to bring to its attention any copyrights, patents, patent applications, or other proprietary rights that may cover technology that may be required to practice this specification.

Author's Address

Daniel Fett

Authlete

Email: mail@danielfett.de